

《信息安全综合实验》实验报告

实验 6：简单入侵检测模型的设计与实现

姓名：蒋衲言 学号：2313546

1 实验目的及要求

实验目的

1. 深入理解特征检测的基本思想。
2. 掌握 K-Means 算法的计算过程。
3. 掌握数据挖掘的基本概念与方法。

实验要求

1. 使用 K-Means 算法对 KDD Cup 1999 数据集进行聚类分析，建立简单的入侵检测模型。
2. 利用入侵检测模型对测试数据进行预测。

2 实验过程

2.1 实验环境与数据集

本次实验在 Linux Ubuntu 虚拟机上进行。

下载得到数据集 kddcup.data_10_percent.gz 与 corrected.gz, kddcup.data_10_percent.gz 作为训练数据集，用于提取数据特征，生成数据挖掘模型；corrected.gz 作为测试数据集，用于验证模型的效率与正确性。

执行命令：

```
gunzip kddcup.data_10_percent.gz
gunzip corrected.gz
```

解压 gzip 压缩过的文件，得到 kddcup.data_10_percent 和 corrected。

2.2 数据预处理

KDD Cup 1999 数据集来源于模拟美国空军军事网络的局域网流量，记录连续 9 周原始 TCP dump 数据，约 700 万条连接记录。每条记录包含 41 项属性，本实验使用其中 18 项。所有记录分为正常（normal）与攻击（attack）两大类。其中攻击行为又分为以下 4 类：

- (1) DOS：拒绝服务，例如，SYN 洪泛攻击。
- (2) R2L：来自于远程主机的非法入侵，例如，猜测密码。
- (3) U2R：非法获得本地主机的超级用户权限，例如，各种“缓冲区溢出”攻击。
- (4) Probing：监视与探测，例如，端口扫描。

故需要对数据集进行预处理。在 DataPreTreat 文件夹下执行以下命令，如图1、2所示：

```
# 编译代码得到可执行文件DataPreTreat
gcc DataPreTreat.cpp -o DataPreTreat
# 预处理kddcup.data_10_percent数据集，得到kddcup.data_10_percent_datatreat
./DataPreTreat ../kddcup.data_10_percent
# 预处理corrected数据集，得到corrected_datatreat
./DataPreTreat ../corrected
```

```
jry@jry-VMware-Virtual-Platform:~/ABC/DataPretreat$ g++ DataPreTreat.cpp -o DataPreTreat
jry@jry-VMware-Virtual-Platform:~/ABC/DataPretreat$ ./DataPreTreat ../corrected
Start reading records from ../corrected...
----- 10000    lines have read -----
----- 20000    lines have read -----
----- 30000    lines have read -----
----- 40000    lines have read -----
----- 50000    lines have read -----
----- 60000    lines have read -----
----- 70000    lines have read -----
----- 80000    lines have read -----
----- 90000    lines have read -----
----- 100000   lines have read -----
----- 110000   lines have read -----
----- 120000   lines have read -----
```

图 1: 预处理数据集

```
jry@jry-VMware-Virtual-Platform:~/ABC/DataPretreat$ ./DataPreTreat ../kddcup.data_10_percent
----- 440000   lines have written -----
----- 450000   lines have written -----
----- 460000   lines have written -----
----- 470000   lines have written -----
----- 480000   lines have written -----
----- 490000   lines have written -----
All Records have written !
```

图 2: 预处理数据集

这一步骤将符号型字段转换为枚举整数值：

- ProtocolType (协议类型): icmp→1, tcp→2, udp→3, 其他 →4;
- Service (服务类型): http→7, ftp→6, smtp→15 等 20 种;
- StatusFlag (状态标志): SF→6, REJ→1, S0→4 等 8 种。

将多种攻击类型合并为 5 大类：

- 0 —normal (正常);
- 1 —DoS (拒绝服务): smurf, neptune, back, teardrop, pod, land;
- 2 —Probe (探测): satan, ipsweep, portsweep, nmap;

- 3 —U2R (普通用户提权): buffer_overflow, rootkit, loadmodule, perl,
- 4 —R2L (远程用户登录): ftp_write, guess_passwd, imap, multihop 等。

最终输出文件名 = 输入文件名 + __datatreat 后缀。得到 kddcup.data_10_percent_datatreat 与 corrected_datatreat。

2.3 编写 K-means 核心代码

我们需要补充的是 Kmeans.cpp 的 main() 函数代码，如下所示：

Kmeans.cpp

```
int main() {
// ...

// PART1 利用K-Means算法，对训练数据集进行迭代聚类，并创建聚类树

// 创建聚类树
pClusterTree = new ClusterTree();
// 创建KMeans聚类对象
pKMeans = new CKMeans(pClusterTree, ++KmeansID, 1, 19);
// 读取训练集中的记录
if(!pKMeans->ReadTrainingRecords())
{
    cout << "Read Training Records Failed!" << endl;
    return -1;
}
// 用户输入K值或直接将K值设置为5
cout << "Please input K value: ";
cin >> Kvalue;
// Kvalue = 5;
// 将K值记录到日志文件中
cout << "K = " << Kvalue << endl;
outfile << "K = " << Kvalue << endl;
// 运行Kmeans算法
pKMeans->RunKMeans(Kvalue);
// 打印聚类树
pClusterTree->Print();
// 将聚类树打印至日志文件中
pClusterTree->PrintLog();

// PART2 利用聚类树 对测试数据集中的数据进行类型预测，并计算出准确率和混淆矩阵

// 读取测试文件
```

```

pTestRcdList = new list<strMyRecord*>();
if(!ReadTestFile(pTestRcdList))
{
    cout << "Read Test File Failed!" << endl;
    return -1;
}
// 创建一个混淆矩阵
ConfuseMatrix* pCfusMatrix = new ConfuseMatrix();
// 预测正确的记录数设置为0
iRightRcdNum = 0;
// 将检测记录数设置为0
TestRcdNum = 0;
// 遍历测试数据链表，为每一个数据找到聚类中心
Rstfile << "===== Test Results
===== " << endl;
for(TestListIter = pTestRcdList->begin(); TestListIter !=
pTestRcdList->end(); TestListIter++)
{
    pRecord = (*TestListIter);
    TestRcdNum++;
    strTrueLabel = LabelInttoStr(pRecord->iLabel);
    pClusterNode = pClusterTree->FindNearestCluster(pRecord);
    strPreLabel = LabelInttoStr(pClusterNode->GetClusterNodeLabel
    ());
    // 统计分类正确的记录数
    if(pRecord->iLabel == pClusterNode->GetClusterNodeLabel())
    {
        iRightRcdNum++;
    }
    // 更新混淆矩阵
    pCfusMatrix->UpdateValue(pRecord->iLabel, pClusterNode->
    GetClusterNodeLabel());
    // 将分类记录写到result.txt文件中
    Rstfile << "Record " << TestRcdNum << ": True=" <<
    strTrueLabel << " Pred=" << strPreLabel;
    if(pRecord->iLabel == pClusterNode->GetClusterNodeLabel())
        Rstfile << " Correct" << endl;
    else
        Rstfile << " Wrong" << endl;
}
// 打印屏显
cout << "===== Test Summary
===== " << endl;
cout << "Total test records: " << TestRcdNum << endl;

```

```

cout << "Correct predictions: " << iRightRcdNum << endl;
outfile << "===== Test Summary
===== " << endl;
outfile << "Total test records: " << TestRcdNum << endl;
outfile << "Correct predictions: " << iRightRcdNum << endl;
// 计算正确分类率
float fAccuracy = (float)iRightRcdNum / (float)TestRcdNum * 100.0;
cout << "Accuracy: " << setiosflags(ios::fixed) << setprecision(2)
    << fAccuracy << "%" << endl;
outfile << "Accuracy: " << fAccuracy << "%" << endl;
Rstfile << "===== Test Summary
===== " << endl;
Rstfile << "Total test records: " << TestRcdNum << endl;
Rstfile << "Correct predictions: " << iRightRcdNum << endl;
Rstfile << "Accuracy: " << fAccuracy << "%" << endl;
// 打印混淆矩阵
pCfusMatrix->PrintMatrix();
pCfusMatrix->PrintMatrixToLog();
}

```

补充的代码主要内容如下：

PART1 —使用 K-Means 对训练数据聚类建模

1. 创建聚类树 ClusterTree;
2. 创建 KMeans 对象（使用构造函数 1, Level=1, 19 维);
3. 读取训练数据—从 kddcup.data_10_percent_datatreat 读取;
4. 用户输入 K 值—通过 cin 读取;
5. 运行 K-Means 算法—RunKMeans(Kvalue) 递归构建层次聚类树;
6. 打印聚类树到控制台和日志文件。

PART2 —使用聚类模型对测试数据预测

1. 读取测试文件—调用 ReadTestFile() 读取 corrected_datatreat;
2. 创建混淆矩阵 ConfuseMatrix;
3. 遍历每条测试记录, 调用 FindNearestCluster() 找到最近聚类节点, 获取其多数标签作为预测标签;
4. 统计正确数, 更新混淆矩阵;
5. 将每条结果写入 Result.txt;

6. 计算并打印准确率和混淆矩阵。

执行以下命令编译：

```
g++ Kmeans.cpp ClusterTree.cpp -o Kmeans
```

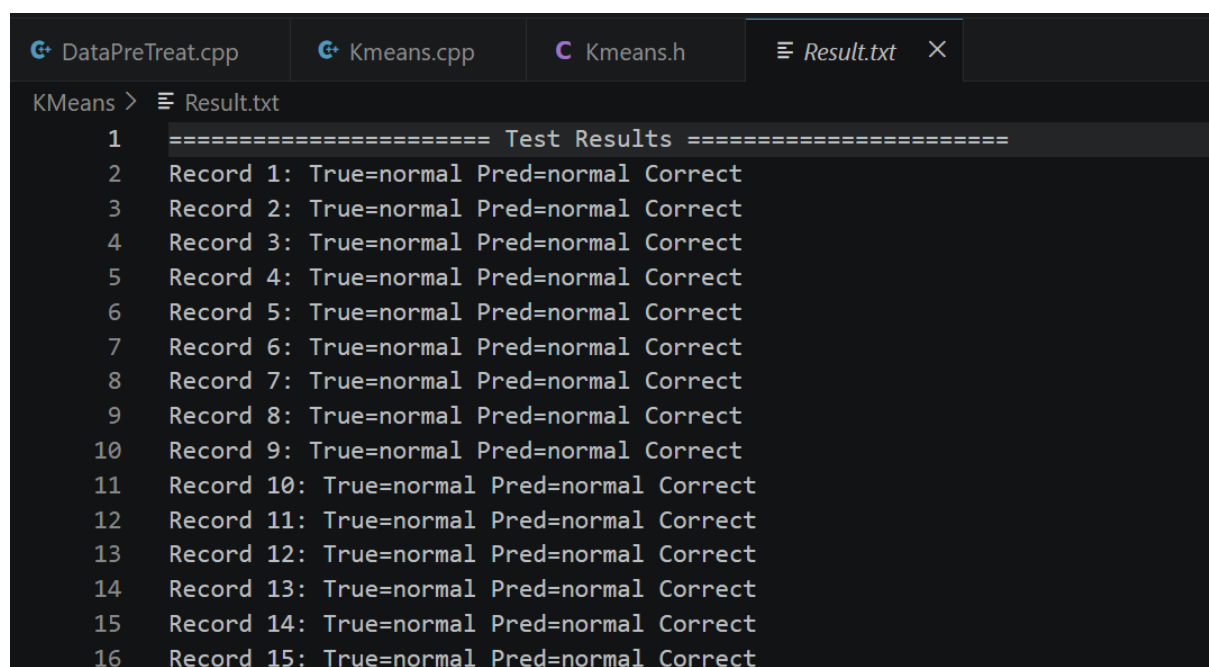
编译得到可执行文件 Kmeans。

2.4 分类结果

将 kddcup.data_10_percent_datatreat 和 corrected_datatreat 放到 Kmeans 同一目录。执行以下命令运行 Kmeans，并输入 K 值为 5：

```
./Kmeans
```

分类结果如图3所示。



```
KMeans > Result.txt
1  ===== Test Results =====
2  Record 1: True=normal Pred=normal Correct
3  Record 2: True=normal Pred=normal Correct
4  Record 3: True=normal Pred=normal Correct
5  Record 4: True=normal Pred=normal Correct
6  Record 5: True=normal Pred=normal Correct
7  Record 6: True=normal Pred=normal Correct
8  Record 7: True=normal Pred=normal Correct
9  Record 8: True=normal Pred=normal Correct
10 Record 9: True=normal Pred=normal Correct
11 Record 10: True=normal Pred=normal Correct
12 Record 11: True=normal Pred=normal Correct
13 Record 12: True=normal Pred=normal Correct
14 Record 13: True=normal Pred=normal Correct
15 Record 14: True=normal Pred=normal Correct
16 Record 15: True=normal Pred=normal Correct
```

图 3: 分类结果 Result.txt

运行输出如图4所示。

```

问题  输出  调试控制台  终端  端口

===== Test Summary =====
Total test records: 311029
Correct predictions: 299950
Accuracy: 96.44%
===== Confusion Matrix =====
T/P  0      1      2      3      4
0    74775  3596  905    0     46
1    143    223136 19     0     0
2    218    120    2039   0     0
3    31     4      0      0     4
4    5442   535    16     0     0
jry@jry-VMware-Virtual-Platform:~/ABC/KMeans$

```

图 4: 运行输出

结果的整体指标如下表所示：

指标	数值
测试记录总数	311,029
预测正确数	299,950
准确率	96.44%

混淆矩阵如下，行（T）为真实标签，列（P）为预测标签：

T/P	0	1	2	3	4
0	74775	3596	905	0	46
1	143	223136	19	0	0
2	218	120	2039	0	0
3	31	4	0	0	4
4	5442	535	16	0	0

各类别召回率分析如下表所示：

类别	总数	正确数	召回率	主要误判去向
normal (0)	79322	74775	94.3%	误判为 r2l (5442) 和 dos (3596)
dos (1)	223298	223136	99.93%	极少误判
probe (2)	2377	2039	85.8%	误判为 normal (218)
u2r (3)	39	0	0%	几乎全部误判为 normal (31)
r2l (4)	5993	0	0%	几乎全部误判为 normal (5442)

可以得出以下结论：

1. **DoS 攻击检测效果极好 (99.93%)**: 该类攻击特征明显, 与正常流量差异大, 聚类算法能有效区分。
2. **Normal 和 Probe 检测较好 (94.3% / 85.8%)**: 两者存在一定程度的特征混淆, 但总体可接受。
3. **U2R 和 R2L 检测失败 (0%)**: 这两类攻击在测试集中样本极少 (U2R 仅 39 条, R2L 仅 5,993 条), 且特征分布与 normal 流量高度相似, 导致 K-Means 聚类无法有效区分。这也是 KDD Cup 99 数据集上的经典难题。
4. **Normal 误报率偏高**: 共约 $3596 + 905 + 5442 = 9943$ 条正常流量被误报为攻击, 说明模型倾向于“宁可误报, 不可漏报”的分类策略。

3 心得体会

本次实验基于 KDD Cup 1999 数据集, 采用层次 K-Means 聚类算法构建入侵检测模型。首先对预处理后的训练集 (kddcup.data_10_percent_datatreat) 进行聚类, 通过递归划分不纯簇, 构建层次聚类树; 然后在包含 311029 条记录的测试集 (corrected_datatreat) 上进行预测。实验结果表明, 模型整体准确率达到 96.44%, 其中 DoS 攻击检测召回率高达 99.93%, 正常流量和 Probe 攻击分别为 94.3% 和 85.8%, 但 U2R 和 R2L 两类攻击的召回率为 0%, 原因在于其训练样本极少且特征与正常流量高度相似, 导致聚类算法难以学习到有效的决策边界, 这也是该数据集上的经典挑战。